

1. Fundamental Concepts

Q1: What is the fundamental difference between **top-down** and **bottom-up** parsing in terms of parse tree construction?

Q2: Define **handle**. In bottom-up parsing, why is identifying the correct handle crucial?

2. Shift-Reduce Parsing

Q3: List and briefly explain the four possible actions in a shift-reduce parser.

Q4: What is a **shift-reduce conflict**? Give a small grammar example that causes it.

Q5: What is a **reduce-reduce conflict**? Provide an example.

3. LR Parsing

Q6: What does **LR(k)** stand for? Explain the meaning of each component (L, R, k).

Q7: Why is **LR(1)** more powerful than **SLR(1)**?

Q8: What is the **canonical collection of LR(1) items**? How does it differ from LR(0) items?

4. LALR vs. LR

Q9: How does an **LALR(1)** parser differ from an **LR(1)** parser in terms of parse table size and power?

Q10: Can an LALR(1) parser have conflicts where LR(1) does not? If yes, why?

5. Practical Parsing

Q11: Why is **LALR(1)** more commonly used in practice than **LR(1)** for programming languages?

Q12: In bottom-up parsing, how are **syntax errors** detected? Compare with top-down error detection.

6. Viable Prefixes & Valid Items

Q13: Define **viable prefix**. Why is the LR parsing algorithm guaranteed to never shift an invalid symbol onto the stack?

Q14: What is the significance of the **LR(0) automaton** in constructing SLR(1) tables?

7. Ambiguity and Conflicts

Q15: Can an ambiguous grammar be used in LR parsing? If so, how are conflicts resolved?

Q16: Explain how **operator precedence** and **associativity** can be used to resolve shift-reduce conflicts in LR parsing.

Q5: What distinguishes local optimization from global optimization?

A5: Local optimization works within basic blocks (straight-line code), while global optimization spans multiple blocks across control flow. Global optimization is more powerful but computationally harder.

Q6: What is the principle behind common subexpression elimination?

A6: *It identifies identical expressions computed multiple times without intervening changes. The computation is performed once, stored in a temporary, and reused, reducing redundant calculations.*

Q7: How does constant folding improve code efficiency?

*A7: Constant folding evaluates constant expressions at compile-time rather than runtime. For example, $x = 3 * 4 + 5$ becomes $x = 17$, eliminating runtime arithmetic operations.*

Q8: What is dead code elimination and how is it identified?

A8: Dead code is code that never executes or whose results are never used. It's identified through control flow analysis (unreachable code) and data flow analysis (unused variables/values).

Q9: What is loop invariant code motion?

A9: *It moves computations outside loops if their operands don't change within the loop. This transforms $O(n)$ computations to $O(1)$, significantly improving performance for large iterations.*

Q10: How does DAG construction help in optimization?

A10: DAGs visually identify common subexpressions, eliminate redundant computations, determine evaluation order, and expose dead code. They transform expression trees into optimized DAGs by merging identical nodes.

Q11: What is strength reduction in optimization?

A11: Strength reduction replaces expensive operations with cheaper equivalents (e.g., multiplication with shift, exponentiation with multiplication). In loops, it replaces multiplication by loop index with addition.

Q12: What are the different levels of code optimization?

A12: Machine-independent (source/IR level), machine-dependent (target code level), local (basic block), and global (inter-block) optimizations. Each level focuses on different optimization opportunities.

Q13: Why can't compilers perform all possible optimizations?

A13: Some optimizations are NP-complete problems, optimization time must be reasonable, aggressive optimization can increase code size, and some transformations may affect debugging or correctness.

What are the primary goals of code optimization in compilers?

A1: The main goals are improving execution speed, reducing code size, minimizing memory usage, and lowering power consumption. Optimization should preserve program correctness and semantics while enhancing performance.

Q2: Differentiate between machine-independent and machine-dependent optimizations.

A2: Machine-independent optimizations work on intermediate code and are portable across architectures (e.g., constant folding, dead code elimination). Machine-dependent optimizations target specific hardware (e.g., register allocation, instruction scheduling).

Q7: What is the purpose of data flow analysis in optimization?

A7: Data flow analysis computes how values flow through the program. It determines reaching definitions, live variables, available expressions, and more. This information enables optimizations like constant propagation, dead code elimination, and register allocation.

Q14: What is register allocation's role in optimization?

A14: Register allocation maps frequently used variables to CPU registers, minimizing expensive memory accesses. It uses techniques like graph coloring to maximize register usage and minimize spills.

Q15: How do basic blocks facilitate optimization?

A15: Basic blocks are maximal straight-line code sequences with single entry/exit. Their simple control flow enables efficient dataflow analysis, common subexpression elimination, and dead code removal.

Q7: What is dominance in CFG and why is it important?

**A7: Node A dominates node B if all paths from entry to B pass through A.*

Dominance identifies loop headers, enables SSA form construction, and supports optimization like loop-invariant code motion and dead code elimination.*

Q8: How are loops identified using CFG?

A8: Loops are identified by detecting back edges (edge from node to ancestor in depth-first order). The natural loop consists of all nodes that can reach the source of back edge without passing through the target's header.

Q4: How do you construct a CFG from three-address code?

A4: Identify leader instructions (first instruction, targets of jumps, instructions after jumps). Form basic blocks from each leader to next leader. Add edges based on jump instructions and fall-through semantics.

Q1: What is a Control Flow Graph and why is it important in compiler construction?

A1: A CFG is a directed graph representing program execution flow. Nodes are basic blocks (straight-line code), edges represent control transfers. It's essential for data flow analysis, optimization, and understanding program structure.

Q2: What defines a basic block in a CFG?

A2: A basic block is a maximal sequence of instructions with single entry (first instruction) and single exit (last instruction). No jumps enter the middle, no jumps exit except at the end. It represents a straight-line code segment.

Q4: Why is register allocation critical in TAC to assembly translation?

A4: TAC uses unlimited temporaries, but processors have limited registers. Efficient register allocation minimizes memory accesses (spills), significantly impacting performance. Poor allocation can negate earlier optimizations.

Q5: What is the difference between local and global register allocation?

A5: Local allocation works within basic blocks, simple but limited. Global allocation spans entire functions using graph coloring, more complex but effective. Global allocation reduces spills across blocks but requires inter-block liveness analysis.

Q1: What are the main challenges in translating 3-address code to assembly language?

A1: The main challenges are register allocation (mapping many temporaries to few registers), instruction selection (choosing optimal machine instructions), addressing mode selection, preserving optimization benefits, and handling architecture-specific constraints like pipeline hazards.

Q2: How does the translation differ between RISC and CISC architectures?

A2: RISC architectures have simple, fixed-length instructions requiring explicit loads/stores (load-store architecture). CISC architectures have complex, variable-length instructions with memory operands. Translation to RISC requires more instructions but simpler instruction selection.

Q3: What is the role of a code generator in the compilation pipeline?

A3: The code generator translates optimized intermediate code (like TAC) to target machine code. It performs instruction selection, register allocation, instruction scheduling, and handles machine-specific optimizations while preserving semantics.

Grammar:

$$\begin{aligned} stmt &\rightarrow \text{while (expr) } stmt \\ stmt &\rightarrow \text{assign} \\ \text{assign} &\rightarrow \text{id = expr ;} \\ \text{expr} &\rightarrow \text{id | num} \end{aligned}$$

1. Write LR(1) states for the production $stmt \rightarrow \text{while (expr) } stmt$.
2. Merge states if possible for LALR(1).
3. Show parsing steps for: **while (id) id = num ;**.

2.

Grammar:

$$\begin{aligned} prog &\rightarrow \text{decl } prog \mid \varepsilon \\ \text{decl} &\rightarrow \text{type idlist ;} \\ \text{type} &\rightarrow \text{int | float} \\ \text{idlist} &\rightarrow \text{id idlist'} \\ \text{idlist'} &\rightarrow , \text{id idlist'} \mid \varepsilon \end{aligned}$$

1. Construct LALR(1) items
2. Show the LALR(1) table entries for **ACTION[state, int]** and **ACTION[state, id]**.
3. Parse: **int a , b ; float c ;**.

3.

Grammar:

$$\begin{aligned} stmt &\rightarrow \text{if (cond) } stmt \text{ else_part} \\ \text{else_part} &\rightarrow \text{else } stmt \mid \varepsilon \\ \text{cond} &\rightarrow \text{expr relop expr} \\ \text{expr} &\rightarrow \text{id | num} \end{aligned}$$

1. Build the LALR(1) parsing states
2. Show the shift/reduce resolution for the *dangling else* in LALR(1) table.
3. Trace parsing for: **if (id < num) if (id > num) stmt else stmt**.

1. $E \rightarrow E1 + T$ { E.type = max(E1.type, T.type);
E.code = gen('+', E1.code, T.code) }
2. $E \rightarrow T$ { E.type = T.type;
E.code = T.code }
3. $T \rightarrow T1 * F$ { T.type = max(T1.type, F.type);
T.code = gen('*', T1.code, F.code) }
4. $T \rightarrow F$ { T.type = F.type;
T.code = F.code }
5. $F \rightarrow (E)$ { F.type = E.type;
F.code = E.code }

6. $F \rightarrow \text{num}$ $\{ F.\text{type} = \text{integer};$
 $F.\text{code} = \text{num.val} \}$
7. $F \rightarrow \text{real}$ $\{ F.\text{type} = \text{real};$
 $F.\text{code} = \text{real.val} \}$

Show the annotated parse tree for the expression:
 $(5 + 3.2) * 4$

Productions with Attributes:

Productions with Attributes:

- $P \rightarrow D ; S$ $\{ S.\text{env} = D.\text{env} \}$
- $D \rightarrow T L$ $\{ L.\text{in_type} = T.\text{type};$
 $D.\text{env} = L.\text{env} \}$
- $T \rightarrow \text{int}$ $\{ T.\text{type} = \text{integer} \}$
- $T \rightarrow \text{float}$ $\{ T.\text{type} = \text{real} \}$
- $L \rightarrow \text{id} = E$ $\{ \text{addtype}(\text{id}.\text{entry}, L.\text{in_type});$
 $\text{checktype}(E.\text{type}, L.\text{in_type});$
 $L.\text{env} = \text{update}(\text{id}.\text{entry}, L.\text{in_type}) \}$
- $L \rightarrow L1 , \text{id} = E$ $\{ L1.\text{in_type} = L.\text{in_type};$
 $\text{addtype}(\text{id}.\text{entry}, L.\text{in_type});$
 $\text{checktype}(E.\text{type}, L.\text{in_type});$
 $L.\text{env} = \text{update}(L1.\text{env}, \text{id}.\text{entry}, L.\text{in_type}) \}$
- $E \rightarrow E1 + T$ $\{ E.\text{type} = \max(E1.\text{type}, T.\text{type}) \}$
- $E \rightarrow T$ $\{ E.\text{type} = T.\text{type} \}$
- $T \rightarrow \text{num}$ $\{ T.\text{type} = \text{integer} \}$
- $T \rightarrow \text{real}$ $\{ T.\text{type} = \text{real} \}$
- $T \rightarrow \text{id}$ $\{ T.\text{type} = \text{lookup}(\text{id}.\text{entry}, S.\text{env}) \}$

Show the **annotated parse tree** for:
`float x = 3.14, y = x + 2.5;`

19. Following are the expression given, you are required to convert these into 3 Address Code, and Directed Acyclic Graph and then convert into Assembly Code.

- $m = (a + b) * c - (a + b) / d$ // (a+b) appears twice
- $n = x * y + z * y + x * w + z * w$ // factorable as $(x+z)(y+w)$
- $p = a * b + b * c + c * a$ // symmetric expression
- $q = (a + b + c) * (a + b + c) - a * a - b * b - c * c$
- $m = (a + b) * c - (a + b) / d$ // (a+b) appears twice
- $n = x * y + z * y + x * w + z * w$ // factorable as $(x+z)(y+w)$
- $p = a * b + b * c + c * a$ // symmetric expression
- $q = (a + b + c) * (a + b + c) - a * a - b * b - c * c$

20. Optimize the Following Code Snippets, Also find the Optimization Technique

1.

```
i = 0
L1: if i >= n goto L2
    t1 = n * 4
    t2 = i * 4
    t3 = arr[t2]
    t4 = t3 * t1
    result = result + t4
    i = i + 1
    goto L1
L2: ...
```

2. i = 0

```
L1: j = 0
    L2: t1 = i * n + j
        t2 = arr[t1]
        j = j + 1
        if j < n goto L2
    i = i + 1
    if i < m goto L1
```

3.

```
t1 = 2 * 3
t2 = a + t1
t3 = 2 * 3
t4 = b - t3
t5 = t2 + t4
t6 = t5 * 1
```

4.

```
call square, (a)
...
# square function:
square(x):
    t1 = x * x
    return t1
```

5.

```
store x, R1
t1 = load R1
t2 = t1 + 5
```



```
store t2, R1
t3 = load R1
t4 = t3 * 2
```

6.

```
t1 = x
t2 = t1 * 5
t3 = y
t4 = t3 + t2
t5 = t1
z = t5 * 2
```

Following are the 3 Address Codes Given, You are required to convert these into Assembly Language..

```
t1 = x * 5
y = t1 + 3
z = t1 * 2
```

```
if a == 0 goto L2
if b == 0 goto L2
result = 1
goto L3
L2: result = 0
L3: ...
```

```
# Original: result = square(x) + square(y)
# After inlining:
t1 = x * x
t2 = y * y
result = t1 + t2
```

```
# After constant folding and propagation
PI = 3.14159
radius = 5.0
t1 = radius * radius
area = t1 * 3.14159
```

Code to 3 Address and machine code examples

```
#include <stdio.h>
int main() {
    int side, i;
    int constant = 1;
    for(i = 1; i <= 3; i++) {
        side = i * 2;
        int volume = side * side * side;
        printf("Volume: %d\n", volume);
    }
    return 0;
}
```

3 address code

```
1. constant = 1
2. i = 1
3. if (i > 3) goto 12
4. t1 = i * 2
5. side = t1
6. t2 = side * side
7. t3 = t2 * side
8. volume = t3
9. param "Volume: %d\n"
10. param volume
11. call printf, 2
12. t4 = i + 1
13. i = t4
14. goto 3
15. return 0
16. end
```

Assembly code

```
main:
    ; Initialize
    mov dword [constant], 1
    mov dword [i], 1
```

```
loop_start:
```

```

; Check condition i <= 3
cmp dword [i], 3
jg loop_end

; side = i * 2
mov eax, [i]
mov ebx, 2
imul ebx
mov [side], eax

; volume = side * side * side
mov eax, [side]
imul eax, [side] ; eax = side * side
imul eax, [side] ; eax = eax * side
mov [volume], eax

; printf("Volume: %d\n", volume)
push dword [volume]
push format_string
call printf
add esp, 8

; i = i + 1
inc dword [i]

; Repeat loop
jmp loop_start

```

```

loop_end:
; return 0
mov eax, 0
ret

```

```

; Data section
section .data
constant: dd 1
i: dd 0
side: dd 0
volume: dd 0
format_string: db "Volume: %d\n", 0

```

2....

```

#include <stdio.h>
int main() {
    int principal, time;

```

```

int rate = 5;
for(time = 1; time <= 3; time++) {
    principal = time * 1000;
    int interest = (principal * rate * time) / 100;
    printf("Interest: %d\n", interest);
}
return 0;
}

```

3 address Code

```

1. rate = 5
2. time = 1
3. if (time > 3) goto 15
4. t1 = time * 1000
5. principal = t1
6. t2 = principal * rate
7. t3 = t2 * time
8. t4 = t3 / 100
9. interest = t4
10. param "Interest: %d\n"
11. param interest
12. call printf, 2
13. t5 = time + 1
14. time = t5
15. goto 3
16. return 0
17. end

```

Assembly Code

main:

```

; Initialize rate = 5
mov dword [rate], 5
mov dword [time], 1

```

loop_start:

```

; Check condition time <= 3
cmp dword [time], 3
jg loop_end

```

```

; principal = time * 1000
mov eax, [time]
mov ebx, 1000
imul ebx
mov [principal], eax

```

```

; interest = (principal * rate * time) / 100
mov eax, [principal] ; eax = principal
mov ebx, [rate]      ; ebx = rate
imul ebx             ; eax = principal * rate
mov ebx, [time]      ; ebx = time
imul ebx             ; eax = principal * rate * time
mov ebx, 100         ; divisor = 100
cdq                  ; prepare for division
idiv ebx             ; eax = eax / 100
mov [interest], eax

```

```

; printf("Interest: %d\n", interest)
push dword [interest]
push format_string
call printf
add esp, 8

```

```

; time = time + 1
inc dword [time]

```

```

; Repeat loop
jmp loop_start

```

```

loop_end:
; return 0
mov eax, 0
ret

```

```

; Data section
section .data
rate:    dd 5
time:    dd 0
principal: dd 0
interest: dd 0
format_string: db "Interest: %d\n", 0

```

```

.3
#include <stdio.h>
int main() {
int radius, i;
int pi = 3;
for(i = 0; i < 4; i++) {
radius = i + 2;
int area = pi * radius * radius;
printf("Area: %d\n", area); }

```

```
return 0; }
```

4.

```
#include <stdio.h>
int main() {
    int num, i;
    int multiplier = 2;
    for(i = 0; i < 3; i++) {
        num = i + 1;
        int square = num * num * multiplier;
        printf("Result: %d\n", square);
    }
    return 0;
}
```

5.

```
#include <stdio.h>
int main() {
    int celsius, i;
    int factor = 9;
    for(i = 0; i < 3; i++) {
        celsius = i * 10;
        int fahrenheit = (celsius * factor / 5) + 32;
        printf("Fahrenheit: %d\n", fahrenheit);
    }
    return 0;
}
```